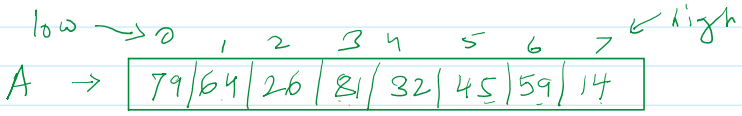
Search techniques

04 July 2026 07:58



Se = 56

Linear search: In linear search we compare the search element with each element of the array. If match is found, we terminate the loop and print/return the index. And, if element is not found then we print the message "element not found in the array" or we return '-1' to the caller of the method.

Linear search works on sorted array or unsorted array also.

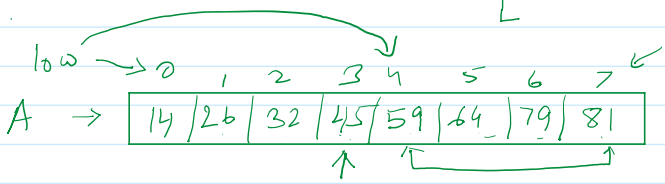
It take <u>linear time</u> to search an element.

Binary Search: Binary search is an efficient algorithm that finds the position of a target value within a sorted array by repeatedly dividing the search interval in half. Unlike a linear search which check elements sequentially, binary search boasts a logarithmic time complexity of $O(\log_2 n)$, making it highly powerful for large datasets.

How it works?

The algorithm uses a divide-and-conquer approach by taking three primary pointers: low, high and mid.

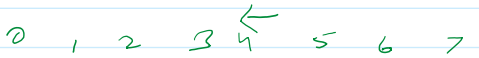
1. Calculate Midpoint: $mid = \left\lfloor low + \frac{high - low}{2} \right\rfloor = 0 + \frac{7-0}{2} = \lfloor 3.5 \rfloor = 3$

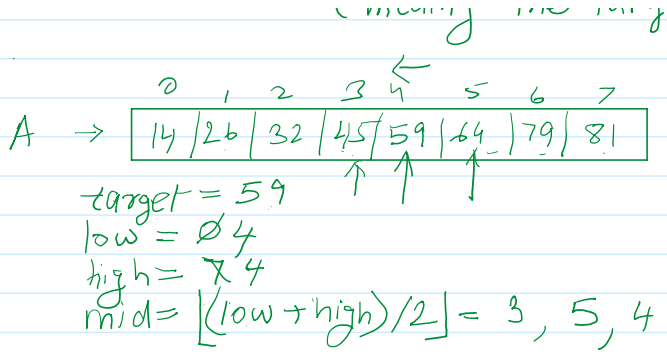


target = 55 / 44

2. Check Target:
   - # if arr[mid] == target, you found it return the index
   - # if arr[mid] < target, the target is in the right half
     set low = mid+1
   - # if arr[mid] > target, the target is in the left half.
     set high = mid-1

3. Repeat: (continue spliting until target is found or low > high (meaing the target is not in the array)






```
package MyArrays;


/**
 * Write a description of class SearchingAlgos here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class SearchingAlgos
{
    public static int binarySearch(int A[], int target){
        int mid, low = 0, high = A.length - 1;
        while(low <= high){
            mid = (low + high) / 2;
            if(A[mid] == target){
                return mid;
            }
            if(A[mid] < target){
                //target may be present in right half
                low = mid + 1;
            }
            else{
                //target may be present in left half
                high = mid - 1;
            }
        }
        return -1;
    }
}
```

```
package MyArrays;
import java.util.*;


/**
 * Write a description of class TestSearchAlgos here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestSearchAlgos
{
    public static void main(String args[]){
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter number of elements: ");
        int n = sc.nextInt();
        int A[] = new int[n];
        ArraysUtility.input(A, "Enter integer values in A[]: ");
        ArraysUtility.printArray(A, "Array A[] before sorting: ");
        SortingAlgos.selectionSort(A, true);
        ArraysUtility.printArray(A, "Array A[] after sorting: ");
        System.out.print("\nEnter your search element: ");
        int target = sc.nextInt();
        int index = SearchingAlgos.binarySearch(A, target);
        if(index > -1){
            System.out.print("\nElement found at index: " + index);
        }
        else{
            System.out.print("\nTarget element is not present in the Array.");
        }
    }
}
```

Output:

